2048 as a Markov Decision Problem: exact dynamic programming on small boards and reinforcement learning on the real game

Project work — Dynamic Programming and Reinforcement Learning, A.Y. 2024/25

Edoardo Caproni (141794) Tommaso Quintabà ([matricola])

November 12, 2026

Abstract

2048 is a puzzle for one player whose rules define a Markov decision problem with a known model. We solve small versions of the game exactly by dynamic programming. Because the sum of the tiles grows at every move, it works as a clock: the game is a total-cost problem that every policy finishes in a bounded number of moves, value iteration becomes exact after finitely many iterations, and ordering the states by tile sum turns it into a single backward pass. The exact solution shows how far the usual rules of thumb are from optimal play (on the 2×3 board with target 64 the corner rule wins 0.07% of the games, the optimal policy 93.9%) and how much of the outcome is luck. We then forget the model and learn from a simulator with the methods of the course, using the exact solution to measure every learned policy without simulation noise. The project started from a reward built on the players' rules of thumb (free cells, a growing tile sum, a bonus for each new largest tile); adding up its parts over a game showed that two of them carry no information about which move is better and the third is a graded win reward, and the exact solution confirmed that it leads to the same play as the game score, which we adopted. In prediction, Monte Carlo beats TD(0) on this long game with a sparse reward, and intermediate n-step TD beats both. In control, tabular Q-learning wins up to 88% of the games and Sarsa with the values written through the afterstates (the board after the move, before the random tile) learns fastest. Carried unchanged to the original 4×4 game, the same methods fill tables with over 20 million boards, nearly all met once, and play no better than the corner rule, with any exploration rate: a table cannot generalise from one board to another. A deep Q-network built as in the PyTorch tutorial is our final test on the original 4×4 board: played on a thousand games at once on a GPU and with a convolutional network, it reaches 17 485 points on average and the 2048 tile in 36% of the games, about six times the score of the rules of thumb but well below the published n-tuple players. We close with what is known about the game: whether it can always be won (open; the best lower bound is 0.99969), whether an optimal strategy is known (no; the problem is NP-hard on general boards), and which methods play it best: the published players that learn afterstate values with n-tuple features, the same update as our best tabular method at a much larger scale.

1 Motivation

2048 (G. Cirulli, 2014 [3]) is a puzzle for one player on a 4×4 grid of numbered tiles. Every move has two steps. First the player slides all the tiles in one direction, which is deterministic. Then a new tile appears in a random empty cell. The rules, including the probabilities of the random tile, are completely known, so the game is a Markov decision problem with a known model: the setting of dynamic programming (DP). The difficulty is size. Small versions of the game (fewer cells, a smaller target tile) can be solved exactly by DP; the original game cannot, because it has far too many states. The whole project is built around this gap.

How the project took shape. Our first idea for teaching an agent to play was to reward it for what good players try to do: keep cells free, make the sum of the tiles grow, and create ever larger tiles. Before training anything with it, we asked what such a reward really asks for once it is added up over a whole game. The answer changed the plan.

- The sum of the tiles grows by the value of the new random tile, whatever the player does (Proposition 1). A reward for the tile sum only rewards surviving; it cannot say which move is better.
- The board is always full when the game is lost, so the changes in the number of free cells add up to the same constant in every game. A reward for free cells cannot change which play is best.
- A bonus for every new largest tile adds up to a function of the largest tile reached. It is a graded version of a simpler goal: create the target tile.

So we stated the goal directly, as the probability of winning (Section 3), and solved it exactly on small boards. The exact solution then became a measuring instrument. It showed that our first reward, the game score and the win itself lead to almost the same optimal play, and that learning from our first reward is no faster than learning from the score, which the game provides for free (Section 6.2). This is the path that led to the two parts of the project.

- **DP part.** On small boards, compute the optimal policy exactly and use it to answer questions that are hard to settle by playing: how well can the game be played, how good are the rules of thumb of human players, and how much of the outcome is luck.
- **RL part.** Forget the model and learn from a simulator, with the methods of the course. On a small board the exact solutions of the DP part measure the learning algorithms precisely. On the original 4×4 board no table fits; there, a deep Q-network built as in the PyTorch tutorial is our final test, and Section 7 reports what others achieved by carrying our best tabular method to the full board.

2 The game

The game is played on a board of $R \times C$ cells with a target tile 2^K. In the original game $R = C = 4$, $K = 11$ (tile 2048) and $p_4 = 0.1$.

1. A move $u \in \{\mathsf{down}, \mathsf{left}, \mathsf{right}, \mathsf{up}\}$ slides all the tiles as far as possible in that direction. Two tiles with the same value that collide merge into one tile carrying their sum. A tile created by a merge cannot merge again in the same move. In each line the merges are resolved starting from the side the tiles move to: moving left, 2 2 2 2 becomes 4 4. The score grows by the value of every tile created by a merge.
2. A move is legal only if it changes the board.
3. After every legal move a new tile appears in an empty cell chosen uniformly at random: a 2 with probability $1 - p_4$, a 4 with probability p_4. The game starts with two such tiles in two different cells.
4. The game is won when a move creates the target tile, and lost when no legal move is left.

3 Formulation as a Markov decision problem

We now write the game in the notation of the DP part of the course [2]: a state x, a control u, a random disturbance w and the dynamics $x_{t+1} = f(x_t, u_t, w_t)$. Table 1 collects the symbols used in the rest of the report.

State. A board is stored as the exponents of its tiles, $x = (x_1, \dots, x_{RC})$, cell by cell in row-major order: $x_i = 0$ for an empty cell and $x_i = k$ for a tile 2^k (so 3 stands for the tile 8). The state x_t is the board the player sees before the t-th move, that is, after the random tile. The initial state x_0 is random, with distribution p_0: two tiles in two different cells chosen uniformly at random, each a 2 with probability $1 - p_4$ or a 4 with probability p_4. The game stops as soon as the target 2^K is created, so before that every cell holds an exponent between 0 and $K - 1$. The state space $\mathcal{X}$ is the set of boards that can be reached from p_0. The game ends in one of two ways: a win, when a move creates the tile 2^K, or a game over, when no legal move is left.

Control. The admissible controls are the legal moves, the ones that change the board:

$$\mathcal{U}(x) = \{u \in \{\mathsf{down}, \mathsf{left}, \mathsf{right}, \mathsf{up}\} : M(x, u) \neq x\},$$

where M is the slide-and-merge map defined next. The game is over exactly when $\mathcal{U}(x)$ is empty.

Deterministic part of a move. $y = M(x, u)$ is the board after the slide and the merges, before the new tile appears. Following [11, §6.8] we call it the *afterstate*: it is the part of the next board that the player controls. M works line by line. Each row (for left and right) or column (for up and down) is read towards the side of the move; the empty cells are removed, adjacent equal tiles are merged from the front, each tile at most once, and the line is padded with empty cells. The points of the move, $r(x, u)$, are the sum of the values of the tiles created by merges.

Disturbance. The random part of a move is $w_t = (i_t, v_t)$: the cell i_t of the new tile, chosen uniformly among the empty cells $E(y_t)$ of the afterstate $y_t = M(x_t, u_t)$, and its exponent v_t, equal to 1 (a 2) with probability $1 - p_4$ and 2 (a 4) with probability p_4. Given the afterstate, w_t does not depend on anything else that happened before. In other words, the move influences the random tile only through the afterstate.

Dynamics. Writing $y + v\, e_i$ for the board y with the exponent v placed in the empty cell i,

$$x_{t+1} = f(x_t, u_t, w_t) = M(x_t, u_t) + v_t\, e_{i_t}.$$

In words: slide, then add the random tile.

Objective. We maximise the probability of creating the target tile:

$$V^*(x) = \max_{\pi}\ P^{\pi}\big(\text{the target tile is created} \mid x_0 = x\big).$$

In the cost form used in the course [2], this is a problem with no cost per stage and a terminal cost of 1 for a lost game and 0 for a won game. Its optimal cost is $J^*(x) = 1 - V^*(x)$, the smallest possible probability of losing. For comparison we also study a second objective: the expected score, with the points $r(x_t, u_t)$ as reward of each stage and no stop at the target tile.

Bellman equation. It helps to split each move into its two steps and give each step its own value. $V^*(x)$ is the best win probability from the board x, before the move. $W^*(y)$ is the best win probability from the afterstate y, before the new tile appears. Then

$$V^*(x) = \begin{cases} 0 & \text{if } \mathcal{U}(x) = \emptyset, \\ \max\limits_{u \in \mathcal{U}(x)} W^*\big(M(x, u)\big) & \text{otherwise,} \end{cases} \tag{1}$$

$$W^*(y) = \begin{cases} 1 & \text{if } \max_i y_i = K, \\ \displaystyle\sum_{i \in E(y)} \frac{1}{|E(y)|} \Big[(1 - p_4)\, V^*(y + e_i) + p_4\, V^*(y + 2e_i)\Big] & \text{otherwise.} \end{cases} \tag{2}$$

In words: (1) says that the player picks the move whose afterstate is worth most, and that a board with no legal move is lost. (2) says that an afterstate holding the target is a win; otherwise its value is the average over where the new tile lands and whether it is a 2 or a 4. For the expected score the move is chosen by $\max_u [r(x,u) + W(M(x,u))]$, which adds the points of the move, and the first case of (2) disappears because the game goes on past the target.

Symbol	Meaning
$R \times C,\ 2^K,\ p_4$	board size, target tile, probability that a new tile is a 4
$x,\ x_t$	board (state) before a move: the exponents of the tiles, 0 for an empty cell
$u,\ \mathcal{U}(x)$	move (control) and set of legal moves
$y = M(x,u)$	afterstate: the board after the slide and merges, before the new tile
$r(x,u)$	points of the move (sum of the tiles created by merges)
$w = (i,v),\ E(y)$	new tile (cell, exponent) and set of empty cells of y
$\sigma(x),\ \mathcal{X}_\sigma$	sum of the tiles; the boards with tile sum σ (a "stage")
$V^*(x),\ W^*(y)$	optimal value of a board and of an afterstate
$Q^*(x,u)$	optimal value of a move, $W^*(M(x,u))$, plus $r(x,u)$ for the score
$\mu,\ \pi$	policy (DP notation and RL notation)
$v_\pi,\ q_\pi$	value of a state and of a state-action pair under π (RL notation)
$\alpha,\ \epsilon,\ \gamma$	step size, exploration rate, discount ($\gamma = 1$ throughout, except DQN)

Table 1: Notation.

4 Structural properties

One simple fact about the rules makes the exact solution easy: the sum of the tiles works as a clock.

Proposition 1 (the tile sum is a clock)**.** *Let $\sigma(x) = \sum_{i:\, x_i > 0} 2^{x_i}$ be the sum of the tiles. For every board x, legal move u and new tile $w = (i,v)$,*

$$\sigma\big(M(x,u)\big) = \sigma(x), \qquad \sigma\big(f(x,u,w)\big) = \sigma(x) + 2^v \in \{\sigma(x)+2,\ \sigma(x)+4\}.$$

Proof. Sliding moves tiles without changing their values, and a merge replaces two tiles 2^k with one tile $2^{k+1} = 2^k + 2^k$. So the slide keeps the sum, and the new tile adds 2^v. □

In words: the move itself never changes the tile sum; only the random tile does, by 2 or 4. The consequences are listed below.

Corollary 1. *(i) The tile sum strictly increases at every move, so no board is visited twice in a game: the graph of the transitions between reachable boards has no cycles.*

(ii) Before the target is created every tile is at most 2^{K-1}, so $\sigma(x_t) \le RC \cdot 2^{K-1}$. Since σ grows by at least 2 per move, every game ends within $RC \cdot 2^{K-2}$ moves, whatever the policy. Hence every policy is proper (it ends the game for sure), and (1)–(2) has a unique solution.

(iii) Group the boards by tile sum: $\mathcal{X}_\sigma = \{x \in \mathcal{X} : \sigma(x) = \sigma\}$. A move only leads from $\mathcal{X}_\sigma$ to $\mathcal{X}_{\sigma+2}$ or $\mathcal{X}_{\sigma+4}$. So V^* can be computed exactly by a single pass backwards over σ, as in the finite-horizon DP algorithm with $\sigma/2$ playing the role of the stage. Value iteration also reaches V^* exactly, after finitely many iterations (Section 5.1).

Link with the theory of the course. In the terms of the course [2], 2048 is an infinite-horizon problem with total cost and two kinds of termination states: the win and the boards without legal moves. Both are made absorbing and cost-free, like the destination in the shortest-path example (a fictitious self-loop with zero cost). Corollary 1(ii) gives the two assumptions of the total-cost

case: termination states exist (A2), and every policy reaches them from every state (A3), here with probability one and within $RC \cdot 2^{K-2}$ moves. So the Bellman equation (1)–(2) has a unique solution, and value iteration converges to it from $V^{(0)} = 0$ without any discount.

Remark (afterstates). A move splits into a deterministic part that the player controls, $x \mapsto y = M(x, u)$, and a random part that the player does not control, $y \mapsto x'$. This is why the expectation over the new tile can be computed in two steps, as in the slides of the course: first the afterstate of every move, then the average over the new tile in (2). The same split gives the RL part an exact way to write the value of a move, $q(x, u) = r(x, u) + V(M(x, u))$ (Section 6.4).

Remark (symmetries). Rotating or reflecting the board (8 ways for a square board, 4 for a rectangular one), and renaming the moves accordingly, gives the same problem. So V^* takes the same value on a board and on its images. The DP does not use this to save work; it uses it as a test of correctness.

5 Exact dynamic programming

On the small boards we compute the optimal policy exactly. Corollary 1(iii) tells us how: list the reachable boards stage by stage, by increasing tile sum, and then solve the Bellman equation backwards, from the last stage to the first. Algorithm 1 is this procedure.

Algorithm 1 Exact DP by stages

Forward pass: starting from the initial distribution (two tiles), generate the reachable sets $\mathcal{X}_\sigma$ for increasing σ
for $\sigma = \sigma_{\max},\ \sigma_{\max} - 2,\ \ldots,\ \sigma_{\min}$ **do** ▷ backward pass
  for each $x \in \mathcal{X}_\sigma$ **do**
    compute $W^*(M(x, u))$ for every $u \in \mathcal{U}(x)$ with (2) ▷ uses V^* on $\mathcal{X}_{\sigma+2} \cup \mathcal{X}_{\sigma+4}$
    $V^*(x) \leftarrow \max_u W^*(M(x, u))$, $\mu^*(x) \leftarrow$ a maximising u ▷ $V^*(x) = 0$ if $\mathcal{U}(x) = \emptyset$
  end for
end for
return μ^* and the value at the start $\sum_x p_0(x)\, V^*(x)$

When a board is processed, every board it can lead to has a larger tile sum and already has its final value; this is why one pass is enough. The same procedure also gives the exact value of any fixed policy (policy evaluation): replace the maximisation with the move that the policy chooses. We use this for the rules of thumb below and, in the RL part, for every learned policy. Ties between moves are broken in the fixed order down, left, right, up. The work is proportional to $\sum_\sigma |\mathcal{X}_\sigma| \cdot |\mathcal{U}| \cdot 2RC$: for every board, every move, and every possible new tile.

5.1 Value iteration and the backward recursion

Value iteration is the algorithm of the course for total-cost problems. It starts from $V^{(0)} = 0$ and, at every iteration, applies the Bellman equation to all the states at once:

$$V^{(i+1)}(x) = \max_{u \in \mathcal{U}(x)} W^{(i)}(M(x, u)),$$

where $W^{(i)}$ is (2) computed with $V^{(i)}$ in place of V^*. It stops when $\max_x |V^{(i+1)}(x) - V^{(i)}(x)| \le \delta$. Started from zero, $V^{(i)}(x)$ has a simple meaning: the best probability of winning within i moves. Since every game ends within a bounded number of moves, the iteration becomes exact after finitely many steps (at most one per stage), not only in the limit.

Table 2 confirms this with $\delta = 0$. On every board value iteration stops at a fixed point that coincides with the backward recursion to the last bit: after 46 iterations on the 2×3 board with target 64 (55 stages), and after fewer iterations than stages on the other boards. Figure 1 shows

how it gets there. The value at the start of the game, the probability of winning within i moves, is zero until $i \approx 27$ and reaches V^* at $i \approx 35$, about the length of an optimal game (32.7 moves on average, Section 5.4). The largest error over the states stays close to 1 for 30 iterations and then falls to zero in 16.

The backward recursion of Algorithm 1 is the same iteration with two changes: the states are updated in place (Gauss–Seidel) and in decreasing tile sum. Each update then uses values of later stages that are already final, so the first sweep gives V^* and the second only confirms it (Table 2). It does the work of one iteration instead of 47 on the 2×3 board, about 55 times less time. This is why we solve the larger boards of Table 3 with it.

Board	Objective	$\lvert\mathcal{X}\rvert$	Stages	Value iteration		$\max_x \lvert V^{\mathrm{VI}} - V^*\rvert$	Gauss–Seidel
				iterations	time (s)		sweeps
2×2, target 16	win	214	11	10	0.01	0	2
2×2, target 32	win	422	19	17	0.03	0	2
2×3, target 32	win	11 222	33	24	1.5	0	2
2×3, target 64	win	27 022	55	47	8.8	0	2
2×3, target 128	win	52 486	93	72	25.7	0	2
2×2	score	662	29	25	0.08	0	2
2×3	score	85 844	125	119	71.8	0	2

Table 2: Value iteration (Jacobi, from $V^{(0)} = 0$, stopping when $\max_x \lvert V^{(i+1)} - V^{(i)}\rvert = 0$) against the backward recursion. Iterations include the last one, which only confirms the fixed point. The difference from the backward recursion is the largest over all the states. Gauss–Seidel sweeps are counted in decreasing tile sum. The backward recursion takes 0.16 s on the 2×3 board with target 64 and 0.51 s with the score objective. For the score objective the game goes on past the target, so the target plays no role.

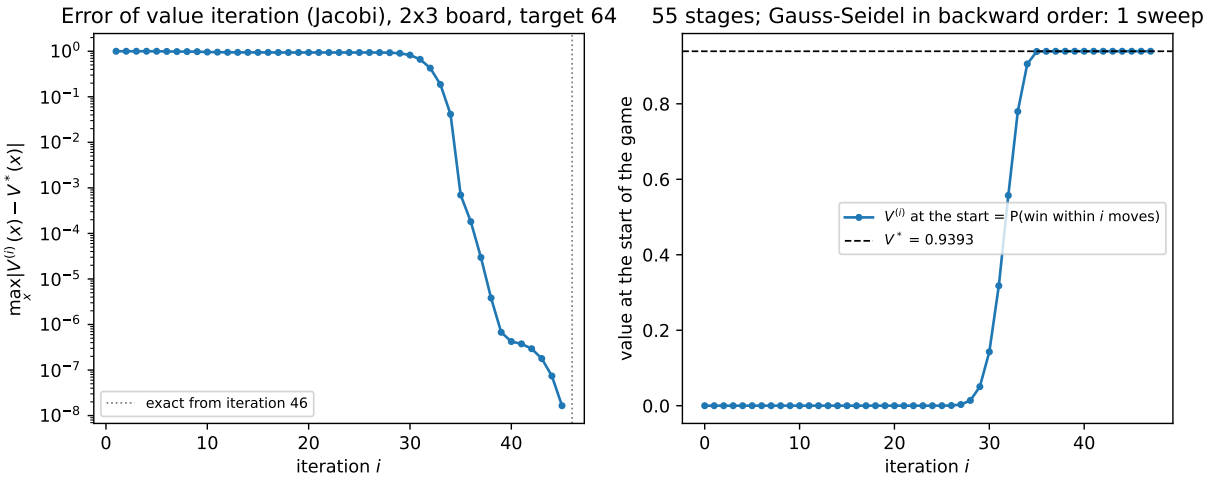


Figure 1: Value iteration on the 2×3 board, target 64. Left: largest error over the states (exactly zero from iteration 46). Right: value at the start of the game, that is, the optimal probability of winning within i moves.

5.2 Benchmark policies

We compare the optimal policy with three simple policies:

- **Corner rule**: the first legal move in the order down, left, right, up. It is the classic rule of thumb: keep the large tiles in one corner and press up only when forced.

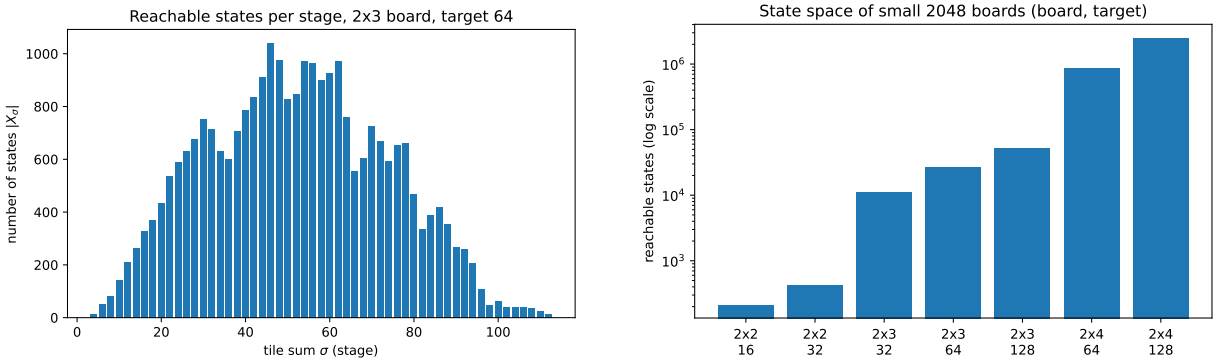
- **Greedy**: the legal move with the largest immediate score $r(x, u)$, ties broken in the same order.
- **Random**: a legal move chosen uniformly at random.

5.3 Size of the state space

Table 3 gives the number of reachable states, together with the simple bound K^{RC} (each cell holds an exponent between 0 and $K-1$). Doubling the target roughly doubles the number of states. Adding cells costs far more: with target 64, the 2×4 board has about 30 times the states of the 2×3 board, and the 3×3 board about 180 times. For the original game the bound is $11^{16} \approx 4.6 \cdot 10^{16}$. Exact DP is out of reach there, and this is why the RL part is needed. Figure 2a shows how the states of the 2×3 board are spread over the stages.

Board	Target	$\lvert\mathcal{X}\rvert$	Bound K^{RC}	DP time	P(win)		
				(s)	optimal	corner	greedy
2×2	16	214	256	0.0	0.9625	0.2355	0.2474
2×2	32	422	625	0.0	0.0829	0.0000	0.0000
2×3	32	11 222	15 625	0.2	0.9999	0.2854	0.3365
2×3	64	27 022	46 656	0.3	0.9393	0.0007	0.0013
2×3	128	52 486	117 649	0.6	0.0570	0.0000	0.0000
2×4	64	881 260	1 679 616	100	0.99999	0.3611	0.3295
2×4	128	2 525 754	5 764 801	272	0.9973	0.0044	0.0037
3×3	64	4 820 069	10 077 696	240	1.0000	0.4341	0.4402
3×3	128	17 244 751	40 353 607	–	–	–	–
4×4	2048	intractable	$\approx 4.6 \cdot 10^{16}$	–	–	–	–

Table 3: Reachable decision states (target not yet created), simple bound, time of the backward recursion in pure Python, and exact win probability of the optimal policy and of two rules of thumb ($p_4 = 0.1$). The random policy wins with probability 0.1218, 0.0005, 0.0921, 0.0002, 0.0000, 0.0274, 0.0000 and 0.0921 on the first eight boards.



(a) States per stage, 2×3 board, target 64.

(b) Reachable states of the small boards.

Figure 2: The curse of dimensionality: the number of states grows exponentially with the number of cells.

5.4 Results on the 2×3 board with target 64

This board is the reference problem of the project. It has 27 022 states in 55 stages, it is solved in a fraction of a second, and its optimal win probability, 0.9393, is neither trivially 0 nor 1. All the

results below are exact unless stated otherwise.

The rules of thumb are hopeless. The corner rule wins 0.07% of the games, the greedy rule 0.13% and random play 0.02%, against 93.9% for the optimal policy. On this board the advice "keep the big tiles in a corner" is worthless. The optimal policy plays the corner move in only 49.5% of its decisions (counting each decision as often as it occurs under optimal play).

Table 4 lists the departures from the corner rule that matter most. They are ranked by how often the board is visited times how much win probability the corner move loses. The boards tell one story: with six cells, what decides the game is not where the largest tile sits, but whether a merge is still available after the next random tile.

- In the first, fourth and sixth boards the corner move keeps the 32 in place but fills the last free cell with tiles that match none of their neighbours. The game then continues only if the new tile is a 4, which happens with probability $p_4 = 0.1$: hence the loss of 0.9. The optimal move merges the same pair in the other direction and leaves a matching pair for the next move.
- In the second and fifth boards the optimal move takes the largest tile *out* of its corner to line up a chain of merges ($4+4$, $8+8$, $16+16$, $32+32=64$ in the second board; in the fifth, the two 32s end up in the same column, an immediate win).
- In the third board the corner rule postpones a merge of two 8s that is available now.

A fixed order of moves cannot see any of this. The optimal policy is not "anti-corner" either: it simply values a merge kept in reserve more than a tile kept in place.

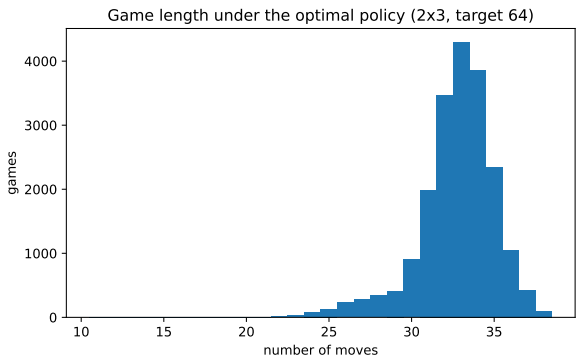
Importance	Board	Corner move	Optimal move	Loss
0.236	$\begin{smallmatrix}16&4&2\\32&8&2\end{smallmatrix}$	down	up	0.90
0.098	$\begin{smallmatrix}32&2&2\\16&8&4\end{smallmatrix}$	left	right	0.90
0.096	$\begin{smallmatrix}8&8&4\\32&\cdot&2\end{smallmatrix}$	down	left	0.86
0.086	$\begin{smallmatrix}16&2&\cdot\\32&8&2\end{smallmatrix}$	right	up	0.90
0.081	$\begin{smallmatrix}2&32&2\\32&4&4\end{smallmatrix}$	left	right	0.91
0.079	$\begin{smallmatrix}16&4&2\\32&2&2\end{smallmatrix}$	down	up	0.86

Table 4: The most important departures of the optimal policy from the corner rule, 2×3 board, target 64. Loss $= V^*(x) - Q^*(x, u_{\text{corner}})$, the win probability given up by the corner move. Importance = expected number of visits of the board under the optimal policy times the loss.

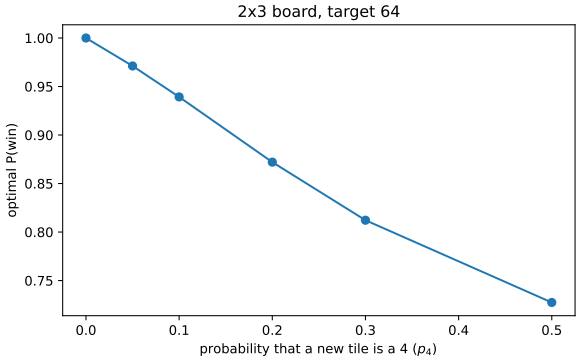
Monte Carlo check. Simulating 20 000 games with the optimal policy gives a win frequency of 0.9395 ± 0.0033 (95% confidence), in agreement with the exact value 0.9393. The games last 32.7 moves on average (Figure 3a).

The role of chance. Figure 3b shows the optimal win probability as a function of p_4. If only 2s appear ($p_4 = 0$), the game is won for sure: the 4s are what makes it lose. At $p_4 = 0.5$ even the optimal player wins only 72.7% of the games. So on this small board luck decides at least 6% of the games, however well one plays.

Win probability or expected score? The same recursion, with the points of each move as stage reward, maximises the expected score: the number the game displays, and what most 2048 programs optimise. The maximum expected score is 480.3. The score-optimal policy wins 93.87% of the games (against 93.93%) and makes the win-optimal move in 97.9% of the decisions. So the



(a) Game length under the optimal policy.



(b) Optimal $P(\text{win})$ as a function of p_4.

Figure 3: Monte Carlo check and sensitivity to the random tile, 2×3 board, target 64.

two objectives are almost, but not exactly, aligned: the score rewards merging, and merging is what makes room for the target. The RL part relies on this: the score is a much denser reward than the win (Section 6.2).

6 Reinforcement learning

6.1 Setting

From now on the agent does not know the model. It only interacts with a simulator of the game, through the `reset`/`step` interface of Gymnasium: it observes the board, chooses a legal move, and receives the next board and a reward. It never sees the probabilities of the random tile. By Corollary 1, every episode ends within a bounded number of moves and no state is visited twice in an episode. So the return without discount ($\gamma = 1$) is well defined, and first-visit and every-visit Monte Carlo methods coincide.

Exact learning curves. On the small boards the DP solution is available, so we do not evaluate the learned policies by simulation but exactly. We take the greedy policy of the agent and compute its exact win probability and expected score with policy evaluation (Section 5.1). We also record two more quantities, both weighted by how often the optimal policy visits each board: the share of decisions in which the agent's greedy move is optimal, and the root mean square error (RMSE) of the learned values against V^*. The resulting curves have no Monte Carlo noise: all their ups and downs come from learning itself.

6.2 Choosing the reward

A learning agent only knows what we want through its reward, so the first design choice is the reward. We describe the path that led to the two rewards used in the rest of the report.

The first idea: reward what good players do. Experienced players keep cells free, see the sum of the tiles grow, and celebrate every new largest tile. Our first reward paid exactly these three things, as a function of the move (x, u) and its afterstate $y = M(x, u)$:

- **milestone**: $2^{\max_i y_i}$ when the move creates a new largest tile ($\max_i y_i > \max_i x_i$), and 0 otherwise. The largest tile never shrinks, so this bonus is paid once for each power of two that becomes the largest tile;
- **tile sum**: $2 + 2p_4$ per move, the expected growth of the tile sum. The real growth is the value of the new tile; we use its expectation so that the reward depends on the move only, as the DP needs;

- **free cells**: $|E(y)| - |E(x)| - 1$, the change in the number of free cells over the whole move (the new tile fills one cell). It equals the number of merges minus one.

We call the sum of the three the *naive reward*. It is a legitimate reward: it depends on the move only, so both the DP and the learning methods accept it.

What the naive reward really asks for. A reward is judged by what it adds up to over a whole game, because that sum, the return, is what the agent maximises. Adding up each part gives three facts.

(1) *The tile sum only rewards surviving.* By Proposition 1 the move never changes the tile sum; only the random tile does. The tile-sum reward is therefore the same for every move: over a game it adds up to $(2 + 2p_4)$ times the number of moves. It says "stay alive", and nothing about which move to make.

(2) *The free cells add up to a constant.* Summing the changes over a game leaves the number of free cells at the end minus the number at the start. A lost game ends on a full board (otherwise a move would be legal), so the sum is $-|E(x_0)|$ in every game, whatever the player does. This part cannot change which play is best: it shifts the value of every board x by exactly $-|E(x)|$.

(3) *The milestones are a graded win reward.* The bonuses of a game add up to about twice the largest tile reached. Paying for the largest tile is a graded version of paying for the target tile, that is, of the win.

Facts (1) and (2) are an instance of a principle stated in the course book [11, §3.2]: the reward should tell the agent *what* we want to achieve, not *how* to achieve it. Free cells and a growing sum are "how"; reaching the target is the "what". Fact (3) points to the "what" directly. We therefore wrote the goal as the probability of winning (Section 3), and measured the naive reward against it.

The measurement. The DP of Part 1 accepts any reward that depends on the move, so on the 2×3 board we computed the optimal policy of each reward exactly and evaluated it on the true goal (Table 5).

- Fact (2) holds exactly. With and without the free-cell part, the optimal policies choose the same move on all 68 340 boards, and the values differ by $|E(x)|$ up to rounding (10^{-13}). On its own the free-cell reward is worth -4 at the start of every game, for every policy: all moves tie, the greedy choice falls back on the fixed order of the moves, which is the corner rule, and wins 0.07% of the games.
- Fact (3) holds too: the policy that maximises the milestones wins 0.9392 of the games, against the best possible 0.9393. It also reaches the tile 128 with probability 0.0568, against the best possible 0.0570 (Table 3).
- The naive reward is not a bad goal: its optimal policy wins 0.9387 of the games, as often as the score-optimal policy. Even the tile sum alone, which only pays for surviving, leads to good play (0.9382), because on this board surviving requires merging well.

Learning with it. The optimum is not the whole story: a learner also needs a reward that tells it something at most moves. We trained the best tabular method of Section 6.4, Sarsa on afterstates, with the naive reward, in the same setting as the other runs (5 seeds, $\alpha = \epsilon = 0.1$, zero initial values). Figure 4 and Table 6 compare it with the same method trained with the score and with the win. The naive reward learns as fast as the score: slightly behind up to about 20 000 episodes, within the spread of the seeds, and equal afterwards (0.797 against 0.796 after 200 000 episodes). Both are far faster at the start than the win, which pays only at the end of a won game: after 10 000 episodes the win reward is at 0.10, the other two above 0.53.

Reward maximised by the DP	P(win)	Expected score	Optimal decisions
win (the goal)	0.9393	466.5	100%
score (points of the merges)	0.9387	480.3	97.9%
naive (milestone + tile sum + free cells)	0.9387	480.2	96.4%
milestone alone	0.9392	469.9	97.8%
tile sum alone	0.9382	479.9	97.9%
free cells alone	0.0007	94.0	49.5%

Table 5: The optimal policy of each reward, 2×3 board, target 64, evaluated exactly on the true goal: win probability, expected score, and share of decisions (weighted by the visits of the win-optimal policy) in which it makes a win-optimal move. After the target is created, the win-optimal policy continues with the score-optimal policy, which only affects its score.

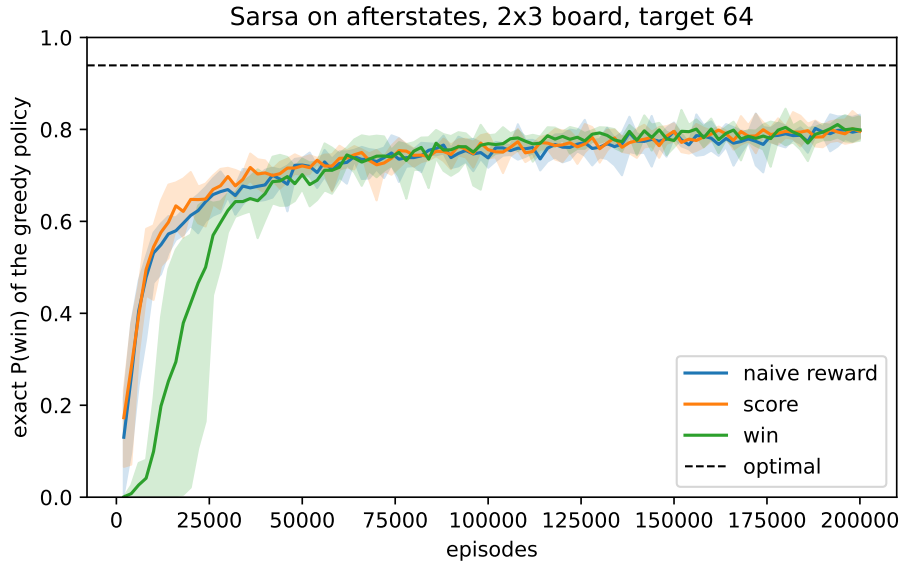


Figure 4: Sarsa on afterstates on the 2×3 board, target 64, trained with three rewards: exact win probability of the greedy policy (mean over 5 seeds, band = min–max). Dashed: optimal.

The decision. The naive reward works, but no better than the score, and the analysis explains why. Of its three parts, two carry no information about which move is better, and the third is a graded win reward. What remains is close to what the score already measures: the score pays for every merge, merging is what keeps the game alive and builds large tiles, and the score-optimal policy makes the win-optimal move in 97.9% of the decisions (Section 5.4). The score has two further advantages: the game provides it, and it has no weights to tune. So in the rest of the report we use two rewards:

- **win**: +1 when a move creates the target tile, which ends the episode, and 0 otherwise. The return of an episode is 1 for a win and 0 otherwise, so the value of a state is its win probability: $v_\pi = P^\pi(\text{win})$, and $v_* = V^*$ of Part 1. This reward is the goal itself, but it is sparse: on the 2×3 board with target 64, random play wins 2 games in 10 000, so an agent that explores at random almost never sees it.
- **score**: the points of each move; play continues past the target and the episode ends at game over. The value is the expected score. It is dense, and almost aligned with the goal.

Whatever the reward used for learning, every policy is measured by its exact win probability.

Episodes	2 000	10 000	20 000	50 000	200 000
naive reward	0.130	0.531	0.613	0.723	0.797
score	0.173	0.544	0.648	0.721	0.796
win	0.001	0.099	0.422	0.702	0.799

Table 6: Exact win probability of the greedy policy of Sarsa on afterstates, mean over 5 seeds, for the three rewards of Figure 4.

6.3 Prediction: TD(0), n-step TD and Monte Carlo

Before learning to play, we study a simpler problem, prediction: estimate the value v_π of a *fixed* policy from its episodes alone. We take the optimal policy of Part 1 and the win reward, so $v_\pi = V^*$, the win probability, is known exactly. This repeats the random-walk examples of the course [11, Example 6.2, §7.1] on a much larger problem: the policy visits 12 939 states instead of 5. We compare three methods, all with a constant step size α and $\gamma = 1$:

- **TD(0)** (§6.1) moves the estimate of a state towards the reward plus the estimate of the next state: $V(S_t) \leftarrow V(S_t) + \alpha\,[R_{t+1} + V(S_{t+1}) - V(S_t)]$;
- **n-step TD** (§7.1) looks n steps ahead before using an estimate: its target is $G_{t:t+n} = R_{t+1} + \cdots + R_{t+n} + V(S_{t+n})$. We use $n = 2, 4, 8, 16$;
- **constant-α Monte Carlo** (§6.1) uses the full return of the game as target. It is the limit $n \to \infty$ of n-step TD.

All the estimates start from $V_0 = 0.5$, as in Example 6.2. The error is $\sqrt{\overline{VE}}$ of §9.2: the RMSE against V^*, weighted by how often the policy visits each state, the same weighting used for the control methods below. Each run simulates 20 000 games of the optimal policy; all the methods learn from the same games; the results are averaged over 50 runs. Since no state repeats within a game, the updates of a game can be applied when it ends, in the order of the online algorithm, with the same result.

Results. Figure 5 reverses the ranking of the random walk, where TD(0) beats Monte Carlo. Here Monte Carlo is the fastest at the start, and TD(0) is the slowest at every budget. Averaged over the first 1 000 games, the best TD(0) error is 0.33 against 0.17 for Monte Carlo; over 20 000 games, 0.114 against 0.085.

The reason is the length of the games and the sparsity of the reward. The only nonzero reward is the win, at the end of a game of about 33 moves. TD(0) moves this information back by only one state per visit, through 12 939 states that are each visited rarely. The Monte Carlo target, instead, carries the outcome to every state of the game at once.

With a longer budget, bootstrapping pays off and an intermediate n wins, as in Figure 7.2 of the book. Over 20 000 games the ranking is:

	8-step	4-step	16-step	Monte Carlo	2-step	TD(0)
average error	0.080	0.082–0.083		0.085	0.095	0.114
best α	0.2			0.1		0.6

With these step sizes the 8-step estimates are also the most accurate at the end: error 0.054 after 20 000 games, against 0.062 for Monte Carlo and 0.064 for TD(0). The best step size decreases as n grows, because a longer return is a noisier target, and all the methods get worse quickly above $\alpha \approx 0.6$. The same trade-off returns in control: the methods that pass information further back at each update learn faster (Section 6.4).

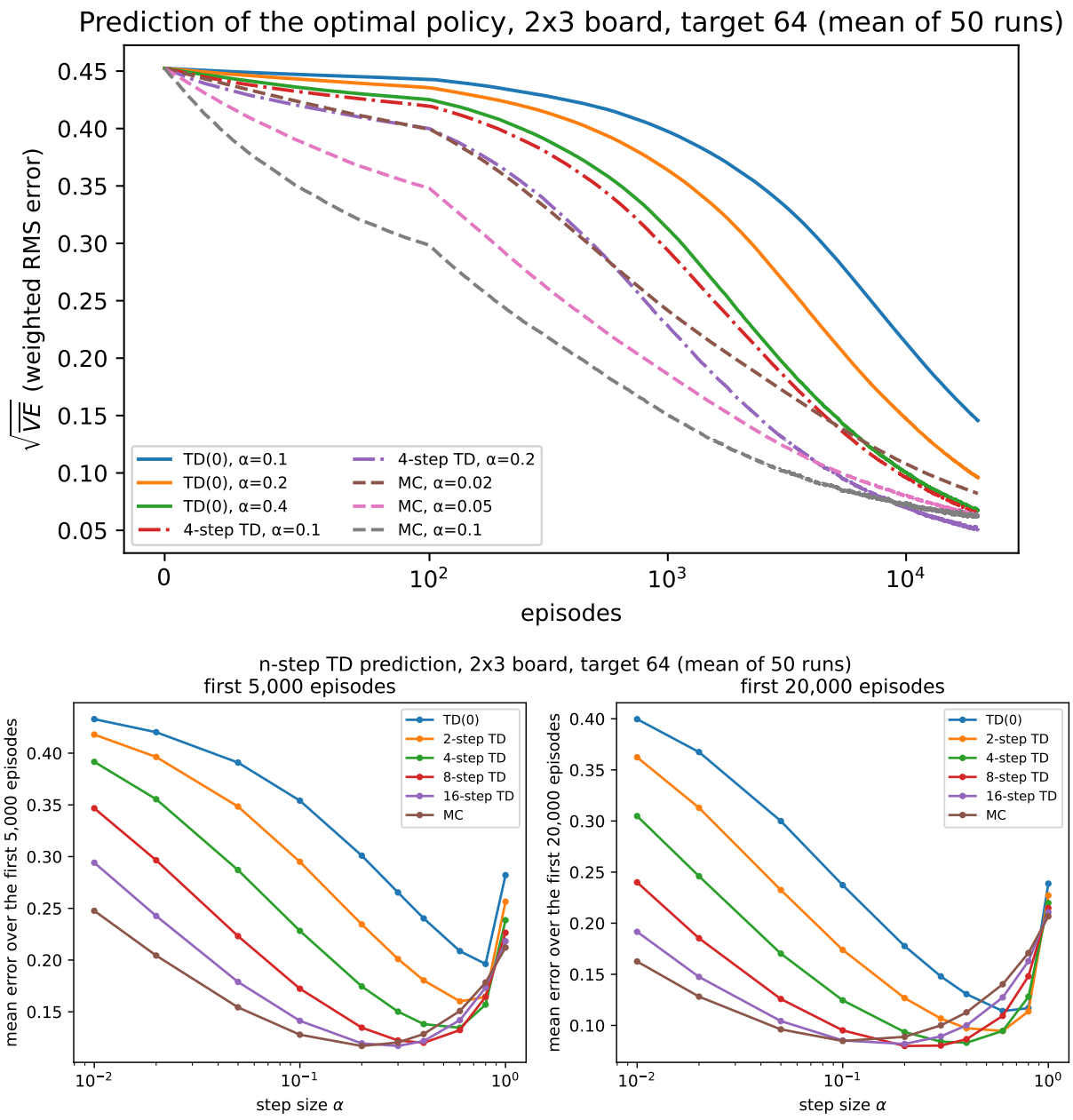


Figure 5: Prediction of the win probability of the optimal policy, 2×3 board, target 64, mean of 50 runs. Top: error against the number of games (logarithmic scale beyond 100). Bottom: error averaged over the first 5 000 and 20 000 games as a function of the step size, one line per n (MC: Monte Carlo); compare [11, Figure 7.2].

6.4 Tabular control on the 2×3 board

Now the agent must learn to play, not just to predict. We implement six control methods of the course [11]. All of them keep a table of values, explore with ϵ-greedy choices among the legal moves, and improve the policy as the values change:

- **Monte Carlo control** (on-policy, first visit, ϵ-soft; §5.4): learns from the full return of each game, with sample averages;
- **Sarsa** (§6.4), **Q-learning** (§6.5) and **Expected Sarsa** (§6.6): one-step TD methods with constant step size α. Sarsa uses the next move actually played; Q-learning the best next move; Expected Sarsa the average over the next moves of the ϵ-greedy policy;
- **n-step Sarsa** (§7.2) with $n = 3$: Sarsa looking 3 moves ahead;
- **Sarsa on afterstates**: Sarsa with the value of a move written through its afterstate, as described next.

Sarsa on afterstates. Two different moves from two different boards can lead to the same afterstate y; from there, the future is the same. So instead of a table $Q(x, u)$ with one entry per board and move, we keep one value $V(y)$ per afterstate and write

$$Q(x, u) = r(x, u) + V(M(x, u)).$$

This is exact, not an approximation: the true value of a move, $q_\pi(x, u) = r(x, u) + \mathbb{E}[v_\pi(x') \mid y = M(x, u)]$, depends on (x, u) only through the points $r(x, u)$ and the afterstate y. The agent needs the slide rule M, but not the probabilities of the random tile. The tic-tac-toe player of [11, §1.5] learns in the same way the values of the positions reached by its own moves. With this Q, the Sarsa error simplifies, because the reward $R_{t+1} = r(S_t, A_t)$ cancels:

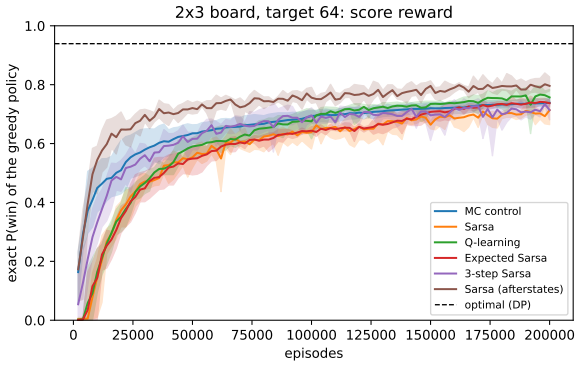
$$R_{t+1} + Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t) = r(S_{t+1}, A_{t+1}) + V(Y_{t+1}) - V(Y_t), \qquad Y_t = M(S_t, A_t).$$

So the Sarsa update of $Q(S_t, A_t)$ becomes an update of one afterstate value, $V(Y_t) \leftarrow V(Y_t) + \alpha\,[r(S_{t+1}, A_{t+1}) + V(Y_{t+1}) - V(Y_t)]$, with target 0 when S_{t+1} is game over. Here A_{t+1} is the next move actually chosen, so the method is on-policy. The same update with a linear approximation of V is the core of the strongest published 2048 players (Section 7).

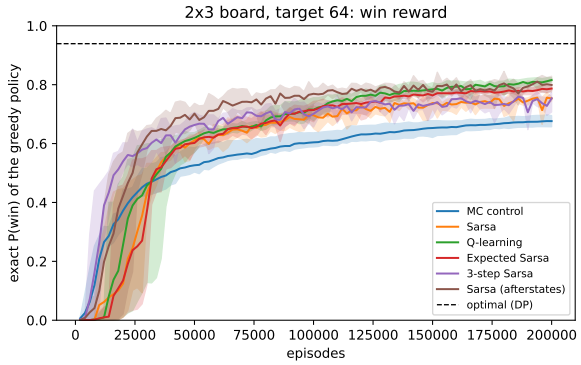
Setup. Every method is run for 200 000 episodes with $\alpha = 0.1$, $\epsilon = 0.1$ and 5 seeds, in three settings: score reward; win reward with zero initial values; win reward with optimistic initial values $Q = 1$ ([11, §2.6]). The greedy policy is evaluated exactly every 2 000 episodes. The greedy choice breaks ties in the order down, left, right, up, so an agent whose values are all zero plays the corner rule.

Results. Figure 6 and Table 7 summarise the 90 runs. Every method learns from the simulator alone to play far better than any rule of thumb. After 200 000 episodes the exact win probability of the greedy policies lies between 0.68 and 0.87, against 0.0007 for the corner rule, and the expected score between 275 and 412, against a maximum of 480. None reaches the optimum 0.9393 within the budget. With constant $\alpha = 0.1$ and $\epsilon = 0.1$ the estimates keep fluctuating, and the exploring policy keeps making random mistakes, which the on-policy methods learn to fear. The ranking agrees with the theory of the course:

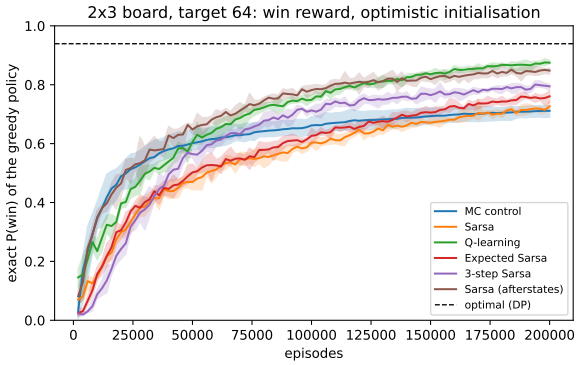
- **Q-learning is the best method on plain tables $Q(x, u)$ in every setting** (0.757, 0.816, 0.875). The reason is that it is off-policy: its target $\max_a Q(x', a)$ estimates the optimal values q_* whatever the exploration, while Sarsa and Expected Sarsa estimate the value of the ϵ-greedy policy, which is lower. The error of the values against V^* shows this clearly. With the win



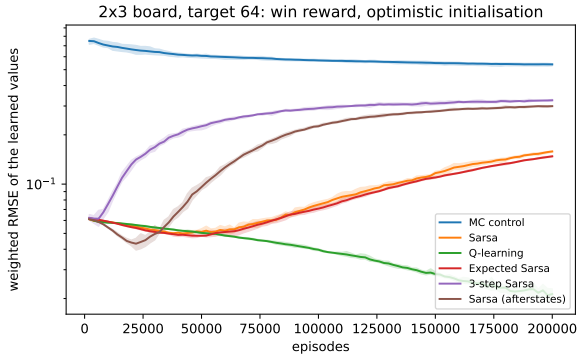
(a) Score reward.



(b) Win reward, zero initial values.



(c) Win reward, optimistic initial values.



(d) Value error, win reward, optimistic initial values.

Figure 6: Tabular methods on the 2×3 board, target 64: exact win probability of the greedy policy (mean over 5 seeds, band = min–max) and weighted RMSE of the learned values.

reward and optimistic initialisation, the weighted RMSE is 0.021 for Q-learning, 0.158 for Sarsa and 0.148 for Expected Sarsa (Figure 6d), although their greedy policies are not far apart. Expected Sarsa beats Sarsa in every setting (its target varies less), and 3-step Sarsa beats both in two settings out of three.

- **Sarsa on afterstates is the best or second best everywhere** (0.796, 0.799, 0.848). It also learns fastest (Figure 6a–c), with the fewest entries: one value per afterstate instead of four per board, 10 478 entries against 12 464 boards seen by Q-learning with the score reward. The shared entries are what make it fast: every update of $V(y)$ improves the value of all the moves, from all the boards, that lead to y. It beats plain Sarsa, the same on-policy algorithm on a plain table, by 0.05–0.12 in every setting, so the gain comes from the way the values are written, not from the algorithm.

- **Monte Carlo control learns fast at first, then slowly.** It leads during the first 20 000 episodes with the score reward, but ends last or second to last. It does not bootstrap, and the returns of 30-move episodes are noisy.

- **The sparse win reward is slower, but not hopeless.** From zero initial values the agents first play the corner rule (the tie-break of the greedy choice). They win nothing for about 10 000 episodes, then take off once ϵ-greedy exploration has stumbled on a few wins whose value spreads backwards (Figure 6b). Optimistic initial values ($Q_0 = 1$) remove this dead start, because every move looks promising until it has been tried. They also leave no state unvisited: the share of the optimal policy's visits to boards the agent has never seen drops from 15–20% to 0. With optimism, Q-learning and Sarsa on afterstates reach their best results. The two other on-policy Sarsa methods gain less: under optimism the exploring behaviour is more erratic, and that is the behaviour they evaluate.

- **The score is a good stand-in for the win.** The greedy policies learned from the score win 71–80% of the games, although they never saw a win signal, in line with Section 6.2.

Step size and exploration. Figure 7 shows the exact win probability after 200 000 episodes of Q-learning and Expected Sarsa, with the score reward, over a grid of constant α and ϵ. Two patterns stand out.

- *Within this budget, larger step sizes win*: $\alpha = 0.2$–0.4 gives 0.80–0.83, against 0.75 for $\alpha = 0.1$ and 0.3–0.5 for $\alpha = 0.02$. The estimates are noisier, but they reach useful values on the 12 000 visited states much sooner, and we evaluate the greedy policy, not the noisy Q.

- *Exploration has an optimum in the middle* ($\epsilon = 0.05$–0.2). With too little, the agent locks onto the first good moves it finds. With too much, the exploring policy loses games, and the on-policy methods learn to avoid the wrong things.

Replacing the constants with decaying schedules (ϵ from 0.3 to 0.01 and α from 0.2 to 0.02, linearly over the run) does not help the one-step methods within this budget (Figure 8). Q-learning ends at 0.72 instead of 0.76, Sarsa and Expected Sarsa at 0.70. The heavy early exploration wastes episodes on positions the greedy policy never visits, and the late small step sizes freeze the estimates before they are accurate. The methods that pass information further back in each episode do gain: 3-step Sarsa rises from 0.71 to 0.76, and Sarsa on afterstates from 0.80 to 0.85, with an exact expected score of 430 out of 480. The schedules that guarantee convergence in the limit (decreasing α, GLIE exploration) are not the fastest for a fixed budget; the right choice depends on how far each update reaches.

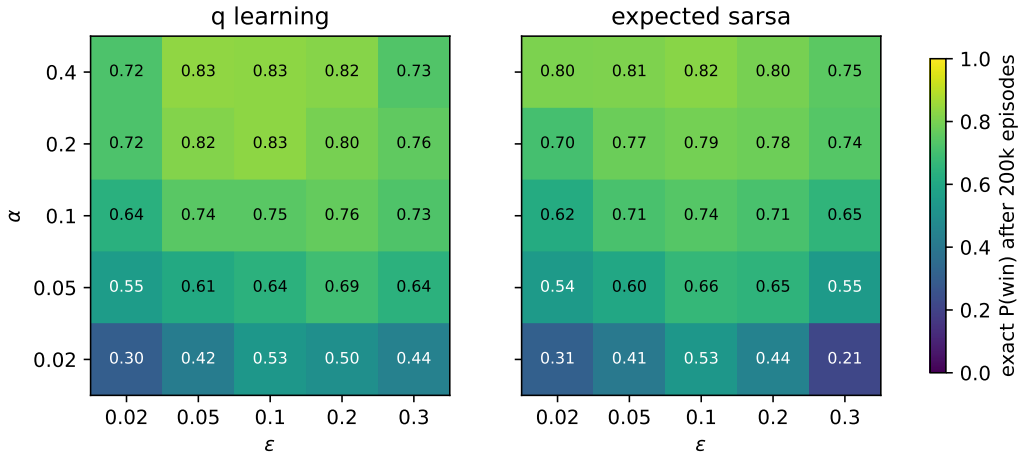


Figure 7: Exact win probability of the greedy policy after 200 000 episodes with the score reward, as a function of the constant step size α and exploration rate ϵ (mean over 2 seeds).

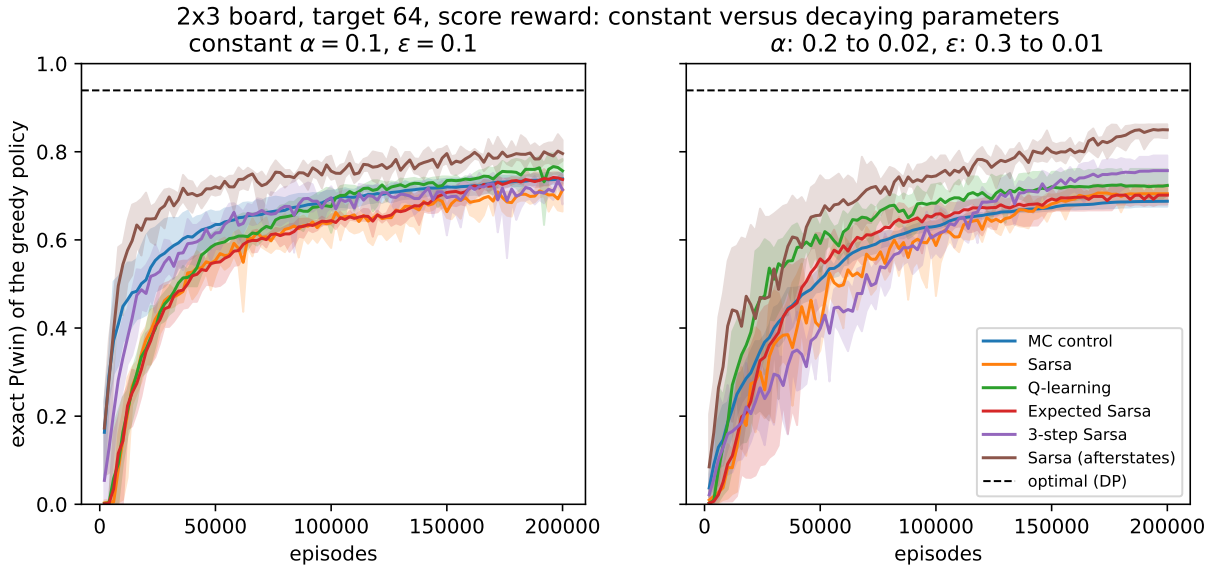


Figure 8: Constant versus decaying α and ϵ, score reward, exact win probability of the greedy policy (mean over 5 seeds, band = min–max).

	score reward			win reward, $Q_0 = 0$			win reward, $Q_0 = 1$		
Method	P(win)	score	agree	P(win)	agree	RMSE	P(win)	agree	RMSE
MC control	0.738	387	0.63	0.676	0.56	0.62	0.711	0.56	0.54
Sarsa	0.713	367	0.53	0.753	0.54	0.68	0.726	0.68	0.16
Q-learning	0.757	383	0.54	0.816	0.56	0.53	0.875	0.78	0.02
Expected Sarsa	0.737	376	0.53	0.787	0.55	0.66	0.761	0.69	0.15
3-step Sarsa	0.714	381	0.59	0.754	0.57	0.62	0.795	0.70	0.33
Sarsa on afterstates	0.796	412	0.63	0.799	0.60	0.62	0.848	0.71	0.30
optimal (DP)	0.939	480	1	0.939	1	0	0.939	1	0

Table 7: Tabular methods on the 2×3 board, target 64, after 200 000 episodes ($\alpha = 0.1$, $\epsilon = 0.1$, mean over 5 seeds; the standard deviation over seeds is at most 0.025). P(win) and score: exact win probability and expected score of the greedy policy. Agree: share of the optimal policy's decisions in which the greedy move is optimal. RMSE: weighted error of the learned state values against V^* (win objective).

6.5 The tabular methods on the original game

Nothing in the six methods depends on the size of the board, so we also ran them on the original 4×4 game, with the settings of Table 7 for the score reward: $\alpha = 0.1$ (sample averages for Monte Carlo control), $\epsilon = 0.1$, zero initial values, 200 000 episodes, 5 seeds. The pure-Python agents cannot hold the tables that this produces, so we ported them to the compiled engine (`rlfast2048.py`). The port changes only the data structure: the table is a hash table indexed by the 64-bit board, and a board enters it when it is met, as in the original code. On the 2×3 board, with the settings of Table 7, the port gives exact win probabilities within the spread of the seeds of the original code (mean of 5 seeds: 0.709 against 0.713 for Sarsa, 0.777 against 0.757 for Q-learning, 0.790 against 0.796 for Sarsa on afterstates). On the 4×4 board the greedy policy of every run plays 1 000 games. On a board that is not in the table there is no value to compare, and the corner rule plays, as in the evaluation of Section 6.4: it is the greedy choice among equal initial values.

The tables do not learn the game. All six methods end below the corner rule (Table 8). The tables are huge but nearly empty of experience: after 200 000 episodes they hold 23 to 28 million boards, of which only about 2 million were met more than once and between 80 000 and 160 000 ten times or more. In a new game 69–74% of the moves are made on boards that the table has never met (50% for Sarsa on afterstates, which needs to know only one of the four afterstates of a board), and the corner rule plays them. The boards that the table does know are mostly those of the first moves of a game, and their values are the returns of one or a few episodes: the moves they suggest are no better than the corner rule. More training does not help (Figure 9). Between 20 000 and 200 000 episodes the tables cover more of the boards met in play (for Q-learning the moves on unknown boards fall from 86% to 68%), but the score does not rise: it stays between 2 100 and 2 500 points for every method.

More exploration makes it worse. With $\epsilon = 0.2$ and 0.3 every method scores less, and the share of moves on unknown boards grows (Q-learning: 2 390, 2 329 and 2 319 points; 69%, 80% and 84%). On the 2×3 board exploration pays because the same boards come back thousands of times, so a move tried once improves an estimate that is used again. On the 4×4 board a board almost never comes back: a random move only takes the training game to boards that the greedy policy will not meet, and ends it sooner (the training games of Q-learning last 173, 149 and 135 moves on average).

What it shows. This is the curse of dimensionality of Table 3, now measured on the learning methods. A table learns each board separately and cannot carry what it learned to a similar board; with up to $4.6 \cdot 10^{16}$ boards, it can learn at most the openings. To play the real game an agent must generalise, that is, approximate the values with a function of a few parameters shared by all the boards (§9.1). The deep Q-network of the next section does it with a neural network, the n-tuple players of Section 7 with a linear function of features of the board.

	$\epsilon = 0.1$			$\epsilon = 0.2$		$\epsilon = 0.3$	
Method	score	unseen	table	score	unseen	score	unseen
MC control	2 287 (2 212–2 350)	74%	28.5 M	2 120	82%	2 129	86%
Sarsa	2 372 (2 311–2 442)	69%	28.2 M	2 319	80%	2 279	84%
Expected Sarsa	2 367 (2 318–2 409)	69%	28.2 M	2 292	79%	2 289	84%
Q-learning	2 390 (2 358–2 440)	69%	28.2 M	2 329	80%	2 319	84%
3-step Sarsa	2 355 (2 305–2 399)	72%	28.3 M	2 207	81%	2 190	85%
Sarsa on afterstates	2 163 (2 147–2 183)	50%	23.6 M	1 958	57%	1 901	62%
corner / greedy rule	2 564 / 3 027						

Table 8: Tabular methods on the 4×4 game after 200 000 episodes: score reward, $\alpha = 0.1$ (Monte Carlo control: sample averages), zero initial values, three exploration rates ϵ; mean over 5 seeds (range in brackets). Score: mean score of the greedy policy on 1 000 games, with the corner rule on the boards not in the table. Unseen: share of its moves on such boards. Table: entries after training (afterstates for Sarsa on afterstates). The rules of thumb: 10 000 games.

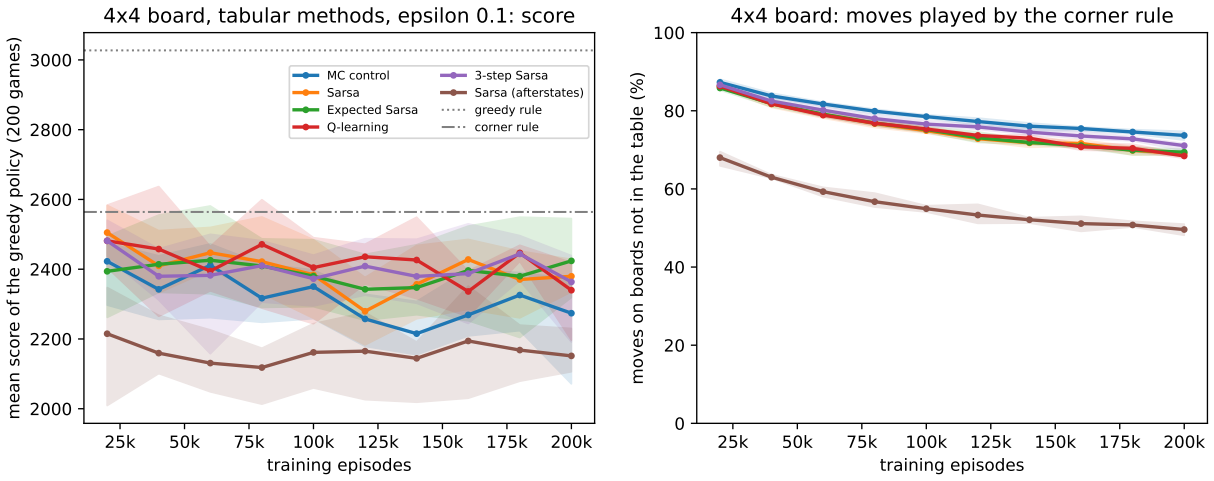


Figure 9: Tabular methods on the 4×4 game, $\epsilon = 0.1$ (mean over 5 seeds, band = min–max). Left: mean score of the greedy policy on 200 games, with the corner rule on the boards not in the table; grey lines: the rules of thumb. Right: share of its moves on boards not in the table, played by the corner rule.

6.6 Final test: deep Q-learning

As a final test, outside the methods of the course, the RL part ends with a deep Q-network (DQN) [8] built as in the PyTorch tutorial [9]. DQN is Q-learning (§6.5) with a neural network approximating the action values (§9.7), stabilised by two devices that the course does not cover: an *experience replay memory*, from which every update draws a random minibatch of past transitions, and a *target network*, a slowly updated copy of the network that computes the targets $r + \gamma \max_{u'} Q'(x', u')$. Network and hyper-parameters are those of the tutorial: a perceptron

with two hidden layers of 128 ReLU units and one output per move; a memory of the last 10 000 transitions and minibatches of 128; the Huber loss; AdamW with learning rate $3 \cdot 10^{-4}$ (amsgrad) and gradients clipped at 100; a soft update of the target network after every move, $\theta' \leftarrow 0.005\,\theta + 0.995\,\theta'$; ϵ-greedy exploration with $\epsilon = 0.01 + 0.89\,e^{-k/2500}$ after k moves; $\gamma = 0.99$; one gradient step per move. Only three things are adapted to 2048: the input is the one-hot encoding of the exponent of every cell (16 values per cell); the illegal moves are masked, both when acting and in the max of the target; the reward is the score of the move divided by the target tile (64 or 2048), so that the returns are of order one, which leaves the greedy policy unchanged. Unlike our Sarsa on afterstates (Section 6.4) and the n-tuple players of the literature (Section 7), the network does not use the slide rule: like the plain tabular Q-learning, it learns $q(x, u)$ for the four moves from the board alone, and it must also discover which patterns of the board matter. We train it with the score reward on the 2×3 board with target 64, where its greedy policy is evaluated exactly, and on the original 4×4 game, evaluated by simulation. There, where a table cannot learn (Section 6.5), the network has to generalise from the boards it has met to the ones it has not.

The schedule of the tutorial is too slow. With one gradient step per move, the tutorial's schedule plays about 550 moves per second on one CPU core: 10 000 games of the 4×4 game take 48 minutes. A GPU does not speed up this loop: with a network this small the time goes into launching the operations and moving the data, not into the arithmetic, and on our laptop GPU the same loop runs at about 480 moves per second. After 10 000 games the DQN scores 2 314 points on average and never creates a 1024 tile, below the corner rule (2 564) and the greedy rule (3 027), although its curve is still rising (Figure 11). A check with the rewards divided by 64 or 256 instead of 2 048 (one seed, 2 000 games) learns a little faster at the start (1 407 and 1 452 points against about 1 100) without changing the picture. On the 2×3 board the same schedule is the fastest learner of the project for a given number of games: its greedy policy wins 0.69 of the games after 2 000 games and 0.812 after 20 000 (Figure 10), while after the same 20 000 games the tabular Q-learning with the score reward is at 0.35 and the Sarsa on afterstates at 0.65. The network generalises across boards, so a game also improves the values of boards it has not visited, and the replay memory reuses every move in about 128 updates.

Many games on the GPU. To play many more games we keep the agent and change only the way it uses its data, following the original DQN [8] instead of the tutorial. 1 024 games are played at the same time: one forward pass on the GPU chooses the moves of all of them, and the rules run in our compiled engine. The replay memory holds the last million moves, as in [8], and every move is replayed 8 times on average, in minibatches of 1 024 (8 times in [8], 128 in the tutorial). Network, loss, optimiser, discount, target network and exploration do not change. Each run now plays about 40 000 moves per second, with the three seeds sharing the GPU, and 100 000 games of the 4×4 game take 18 minutes. On the 2×3 board, after 200 000 games (6 minutes), the greedy policy wins 0.871 of the games (0.868–0.872 over three seeds), with an expected score of 435: above every tabular method trained with the score reward (at most 0.796 after the same number of games), and level with the best tabular run, Q-learning with the win reward and optimistic initial values (0.875). On the 4×4 board, after 100 000 games, it scores 8 862 points on average (8 466–9 328), reaches 1024 in 41% of the games and 2048 in 3%. More games alone are not enough, though: from 40 000 games on the curve stays at about 8 800 points (Figure 11).

A convolutional network. The perceptron has no way to know that a pair of equal tiles matters in the same way in every row and column; a convolutional network knows it by construction. Ours reads the one-hot board as 16 channels on the 4×4 grid. A first layer of 1×2 and 2×1 convolutions with 128 channels reads the horizontal and vertical pairs of neighbouring cells, the pairs that can merge; a second layer applies 1×2 and 2×1 convolutions to each of these maps, reading pairs of pairs. All the maps are joined in a hidden layer of 256 units, which gives the four

action values: 2.0 million weights, against 50 000 for the perceptron. The rest is the schedule of the previous paragraph. The network is slower (about 12 000 moves per second per run, with the two seeds sharing the GPU and its tensor cores in TF32 precision), and 100 000 games took 90–95 minutes. The runs were stopped at 50 000 games and resumed from the saved network, with the replay memory first refilled with a million moves of that network: in a test, restarting from an empty memory cost about 6 500 points. The convolutional network gives the best result of the final test: 14 027 points after 50 000 games and 17 485 after 100 000 (16 184–18 786), with the 1024 tile in 73% of the games, 2048 in 36% and 4096 in 1.3%. With the same number of games it doubles the score of the perceptron and creates the 2048 tile thirteen times as often. The two seeds parted in the second half: one passed 18 000 points at 70 000 games and stayed there, the other oscillated around 15 000 from 60 000 games on, and by 100 000 games neither curve was still rising (Figure 11).

	Games	Moves	Time	Greedy policy
2×3 *board, target 64*				
DQN, schedule of the tutorial	20 000	0.84 M	21 min	exact $P(\text{win}) = 0.812$ (0.807–0.818), score 408
DQN, many games on the GPU	200 000	8.6 M	6 min	exact $P(\text{win}) = 0.871$ (0.868–0.872), score 435
Q-learning, tabular	200 000			$P(\text{win}) = 0.757$ (score reward), 0.875 (win reward)
Sarsa on afterstates, tabular	200 000			$P(\text{win}) = 0.796$ (score reward), 0.848 (win reward)
optimal policy (DP)				$P(\text{win}) = 0.939$, score 480
4×4 *board, target 2048*				
DQN, schedule of the tutorial	10 000	1.5 M	48 min	2 314 points (2 221–2 461); 512 in 4%, never 1024
DQN on the GPU, perceptron	100 000	43 M	18 min	8 862 points (8 466–9 328); 1024 in 41%, 2048 in 3%
DQN on the GPU, convolutional	50 000	26 M	39 min	14 027 points (13 537–14 517); 1024 in 65%, 2048 in 20%
DQN on the GPU, convolutional	100 000	65 M	95 min	17 485 points (16 184–18 786); 1024 in 73%, 2048 in 36%, 4096 in 1.3%
corner / greedy rule				2 564 / 3 027 points; 512 in 5% / 12%

Table 9: Final test: DQN with the score reward (mean over the seeds, range in brackets; three seeds, two for the convolutional network). Time: wall-clock time of one run; the seeds of a configuration ran in parallel, on one CPU core each with the tutorial's schedule and on the same GPU otherwise. 2×3 board: exact evaluation of the greedy policy; the tabular methods with $\epsilon = 0.1$, $\alpha = 0.1$, and optimistic initial values with the win reward (Table 7). 4×4 board: 1 000 games with a fixed seed, 10 000 for the rules of thumb.

What the final test shows. Table 9 and Figures 10–11 summarise the runs. A generic network learns 2048. On the small board, given enough games, it plays as well as the best table; it still stops short of the optimum (0.871 against 0.939, agreement 0.66 with the optimal decisions), because a network of this size cannot represent V^* exactly and because it maximises the discounted score ($\gamma = 0.99$), not the win. On the real game both the amount of data and the network matter: from the tutorial's schedule to many games on the GPU, and from the perceptron to the convolutional network, the mean score grows from 2 314 to 8 862 to 17 485 points, about six times the rules of thumb. It stays well below the n-tuple players of the literature, which reach 2048 in over 97% of the games [12], against 36% here. Those players are given what the DQN has to learn: the slide rule, through the afterstate values $\hat{q}(x, u) = r(x, u) + \hat{V}(M(x, u))$, and features that describe the board in the terms that matter; they also maximise the score without discount, while the

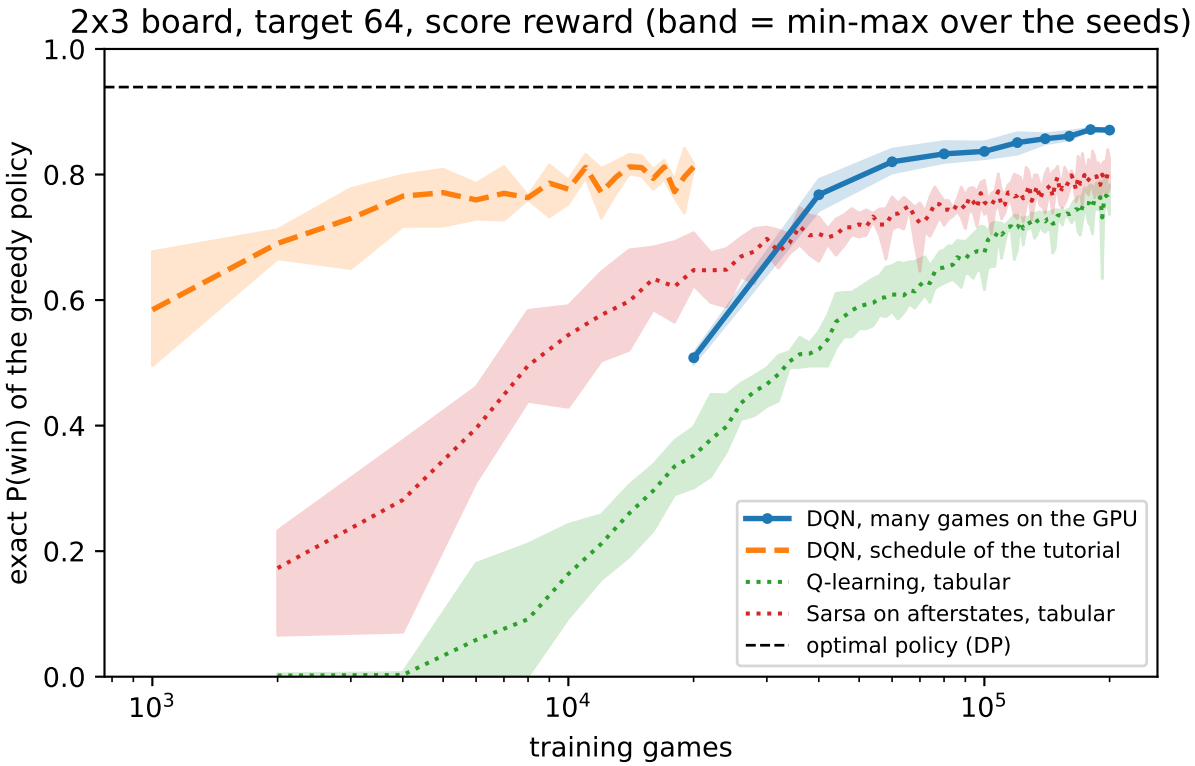


Figure 10: Final test on the 2×3 board, target 64, score reward: exact win probability of the greedy policy against the training games, for the DQN with the tutorial's schedule (three seeds, 20 000 games) and with many games on the GPU (three seeds, 200 000 games), and for the tabular Q-learning and Sarsa on afterstates (five seeds). Band: min–max over the seeds.

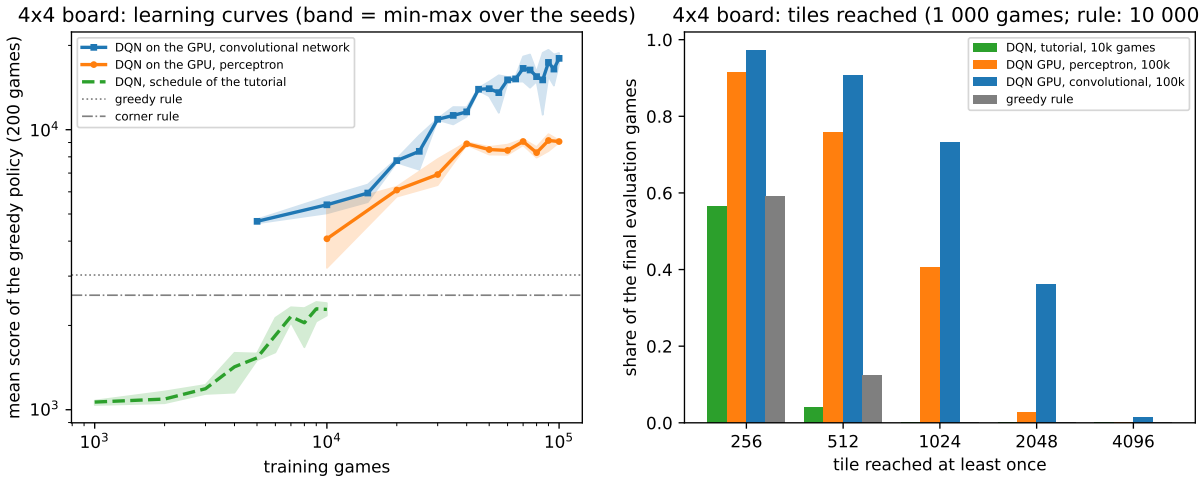


Figure 11: Final test on the 4×4 game. Left: mean score of the greedy policy against the training games (200 games per point), for the DQN with the tutorial's schedule and on the GPU with the perceptron and with the convolutional network, and the two rules of thumb. Right: share of the final evaluation games that reach each tile.

DQN looks about 100 moves ahead in games that now last about 1 000 moves. The convolutional network goes in the same direction as the n-tuple features, since it reads patterns of neighbouring cells everywhere on the board, and that alone doubled the score.

7 The game itself: what is known

A player asks three questions about 2048: can the game always be won, is there an optimal strategy, and which rules of thumb are good. On the small boards our exact results answer all three. For the original game the answers come from the literature, and this section collects them. It also reports what other people achieved on the 4×4 board with the learning methods that go beyond the course, and compares them with our results.

Is 2048 always solvable? Not on the small boards. On the 2×3 board with target 64 even the optimal player loses 6% of the games, and as soon as $p_4 > 0$ there are unlucky sequences of new tiles that no policy survives (Figure 3b). On the 4×4 board the question is open. Slizkov [10] gives the only rigorous results: a strategy that reaches the 256 tile with probability 1, and a strategy that creates the 2048 tile with probability at least 0.99969. Whether some strategy wins with probability 1 is not known. If an adversary placed the new tiles instead of chance, the game would be lost well before 2048: this is easy to believe and to observe in "evil" variants of the game, but we found no proof of it. The largest tile that can ever appear on a 4×4 board is $131072 = 2^{17}$ (sixteen cells holding $2^{17}, 2^{16}, \ldots, 2^2$, plus a lucky final 4), and the best programs reach $2^{15} = 32768$ regularly.

Is the optimal strategy known? No, and it is probably out of reach. The 4×4 game has up to $11^{16} \approx 4.6 \cdot 10^{16}$ states before the target (Table 3), many orders of magnitude beyond exact DP; our exact solutions stop at the 3×3 board with 5 million states. Results from computational complexity show that the difficulty lies in the rules themselves, not only in the size of the board. Langerman and Uno [6] prove that deciding whether a given position on an $m \times n$ board can be played to reach a given tile is NP-hard, for 2048, Threes! and most of their variants. Abdelkader, Acharya and Dasler [1] prove NP-hardness even when no new tiles appear. Mehta [7] shows that reaching a target configuration is PSPACE-complete for a variant of the game. These results are about general board sizes and worst-case positions, not about the probability of winning the 4×4 game, but they rule out a simple closed-form optimal strategy.

Which rules of thumb are good? Human players use the corner rule (keep the largest tile in a corner, and the big tiles along one edge ordered as a "snake"), keep the board monotone and smooth, and keep as many cells free as possible. Our exact results qualify this advice. On the small boards the corner rule is far worse than optimal play: 0.07% against 93.9% of games won on $2 \times 3/64$, 36% against 99.999% on $2 \times 4/64$, 43% against 100% on $3 \times 3/64$. A fixed order of moves wastes the few cells available. On the 4×4 board the rules of thumb alone are weak too: in 10 000 games played with our engine the corner rule scores 2 564 points on average and the greedy rule 3 027; neither ever creates the 2048 tile (the greedy rule reaches 1024 in 0.1% of the games). The same rules become strong when they are combined with search. The expectimax player of [13] looks several moves ahead and scores the positions it reaches with a hand-tuned evaluation (monotonicity, smoothness, free cells, merges). In 100 games it always reached 2048, reached 16384 in 94% of them and 32768 in 36%.

Learning on the real board: afterstates and n-tuple networks. The strongest players do not hand-tune the evaluation: they learn it, with the update of our Sarsa on afterstates (Section 6.4). On the 4×4 board a table with one value per afterstate is impossible, so they replace it with a linear approximation, $\hat{V}(y) = \mathbf{w}^\top \boldsymbol{\phi}(y)$ [11, §9.4], and apply the semi-gradient

form of the same update [11, §10.1]. The features $\boldsymbol{\phi}$ are called an *n-tuple network* [12]. Each feature looks at a fixed group of n cells (a row, a column, a 2×3 rectangle) and reads the pattern of tiles in it; each possible pattern has its own weight in a lookup table. The value of a board is the sum of the weights of the patterns it contains. Table 10 collects the main results.

Method	Result on the 4×4 game
Corner rule / greedy rule (our runs, 10 000 games)	2 564 / 3 027 points on average; never 2048
Expectimax search with a hand-tuned evaluation [13]	2048 in all of 100 games; 16384 in 94%, 32768 in 36%
TD learning of afterstate values with 17 tuples of 4 cells [12]	over 97% of games reach 2048
Multistage TD learning, larger tuples, expectimax search [14]	32768 in 31.75% of 10 000 games; once 65536
Delayed temporal coherence learning, weight promotion, redundant encoding [5]	the best 2048 program known at the time of publication
Optimistic initialisation, multistage learning, expectimax search [4]	32768 in 72% of the games; average score 625 377

Table 10: Published results on the original game, compared with the rules of thumb measured with our engine.

How do these compare with what we did?

- *Same update, different scale.* Szubert and Jaśkowski [12] learn afterstate values with the same update as our Sarsa on afterstates, playing greedily, with n-tuple features in place of the table. Our table can only work on boards small enough to list every afterstate; the features are what carry the same update to the 4×4 board. On the 2×3 board our tabular version wins 80–85% of the games within 200 000 episodes; their 4-tuple network reaches 2048 in over 97% of the games on the real board.

- *Optimism helps in both.* Guei [4] sets all the weights to large initial values to make the agent explore states it has rarely visited. This is the optimistic initialisation of [11, §2.6], and it is also what gave our best tabular results with the sparse win reward (0.875 for Q-learning, 0.848 for Sarsa on afterstates, Table 7).

- *The step size matters.* Our grid showed that the best constant step size depends on the budget, and that decaying schedules do not always help (Figure 8). Jaśkowski [5] avoids the choice with temporal coherence learning, which adapts the step size of each weight automatically.

- *Stages.* Yeh et al. [14] split the game into stages by the largest tile reached and train separate weights for each stage. This echoes our use of the tile sum as a clock (Corollary 1): in both cases, where the game stands is a key piece of information.

These programs add search (expectimax over the next moves and tiles), larger networks and millions of training games. At the core, though, they use an update of the course: what separates them from our tabular agent is the scale of the approximation and of the computation, not the learning rule.

8 Discussion and conclusions

What dynamic programming taught us about the game. The tile sum is a clock. So 2048, on any board, is a total-cost problem whose termination states every policy reaches within a bounded number of moves. Value iteration is therefore exact after finitely many iterations (46 on the 2×3 board), and ordering the updates by tile sum turns it into a single backward pass

over the reachable states. On the boards where this pass is affordable (up to 3×3, five million states), the exact values overturn the folk wisdom of the game. The corner rule, the greedy rule and random play win a fraction of a percent of the games on $2 \times 3/64$ and $2 \times 4/128$, where the optimal player wins 94% and 99.7%; and the optimal policy agrees with the corner rule in only half of its decisions. Table 4 shows why: on a cramped board, what matters is whether a merge survives the next random tile, and a fixed order of moves ignores this. Luck limits every player: as soon as $p_4 > 0$ the optimal win probability is below one, and it is very sensitive to p_4 (Figure 3b). Finally, the exact state counts (Table 3) make the curse of dimensionality concrete: the 4×4 board is many orders of magnitude beyond the last solvable board, and the complexity results of Section 7 say that no shortcut should be expected.

Dynamic programming as a measuring instrument. The second use of the exact solution was to measure. Every learned policy was evaluated exactly instead of by simulation, and every learned value function was compared with V^*. The learning curves therefore have no Monte Carlo noise: differences of a few hundredths between methods are real, and the only randomness is the learning itself. This made visible things that simulation would have blurred:

- the choice of the reward: the rules-of-thumb reward we started from, the score and the win lead to almost the same optimal play (0.9387, 0.9387, 0.9393), and the analysis of what each reward adds up to explains why (Section 6.2);
- on this game Monte Carlo prediction beats TD(0), and intermediate n beat both: the reverse of the textbook random walk at the start;
- the gap between a learned Q and the greedy policy it induces: about 0.6 agreement with the optimal decisions and 0.8 of the optimal win probability at the same time.

What worked in reinforcement learning, and why. The first decision was the reward. Our first idea, rewarding free cells, a growing tile sum and new largest tiles, turned out to contain no more than the game score: two of its parts carry no information about which move is better, and the third is a graded win reward. It learned as well as the score, but no better, so we kept the score, which the game provides with no weights to tune. With this dense reward every tabular method of the course learns a reasonable policy on 2×3 within 200 000 episodes. With the sparse win reward and zero initial values the greedy learner locks onto the first moves that ever paid, and optimistic initialisation is what repairs it, because every move looks promising until it has been tried. Q-learning was the best method on plain tables (0.88 with optimism). Sarsa on afterstates was the best method overall (0.85 with decaying schedules), for a structural reason: writing $q(x, u) = r(x, u) + V(M(x, u))$ is exact, one value per afterstate replaces four values per board, the agent needs the slide rule but not the probabilities of the random tile, and every update improves all the moves that lead to the same afterstate. Larger constant step sizes won within a fixed budget, and decaying schedules only helped the methods that pass information further back at each update: the same trade-off that made an intermediate n best in prediction.

From the small board to the real game. Tables stop at small boards. Carried unchanged to the 4×4 game, the six methods meet 23–28 million boards in 200 000 episodes, nearly all of them once, play 69–74% of their moves on boards they have never met, and stay below the corner rule; more exploration only makes it worse (Section 6.5). The literature (Section 7) shows what happens when our best tabular method, Sarsa on afterstates, is carried to the 4×4 board with a linear approximation and n-tuple features: over 97% of games reach 2048 [12], and with search, optimism and staged learning the 32768 tile is reached in 72% of the games [4]. These players use an update of the course; what separates them from our tabular agent is the scale of the approximation and of the computation.

The final test. The deep Q-network of the PyTorch tutorial, outside the course, learns the small board faster than any table for a given number of games, and with many games on a GPU it plays it as well as the best table (0.871 against 0.875). On the 4×4 game the tutorial's schedule is too slow to get far (2 314 points after 10 000 games, below the rules of thumb); playing a thousand games at once on the GPU, with a larger replay memory, and replacing the perceptron with a convolutional network take it to 17 485 points and the 2048 tile in 36% of the games. The n-tuple players of the literature still do much better (2048 in over 97% of the games), because they are given the slide rule and features made for the game: the same lesson as on the small board, where writing the action values through the afterstates made Sarsa learn fastest.

What remains open. Whether the original game can be won with probability one is unknown (the best lower bound is 0.99969). The optimal policy is out of reach of exact methods, and the problem is NP-hard on general boards, so the strongest strategies are learned approximations, validated by play rather than by proof. Within the methods of the course, the natural next step for this project is the one the literature took: Sarsa on afterstates with a linear approximation on the 4×4 board. The small boards would then allow a measure that the literature cannot make: how far the best value function that a given set of features can represent is from V^*.

A Reproducibility

The code is at `https://github.com/EdoardoCaproni/2048` (Python 3.10 or later, numpy, matplotlib, numba; PyTorch for the final test). The modules are `game2048.py` (rules of the game and the Gymnasium-style environment; the reference implementation), `dp2048.py` (exact DP: reachable states by stage, backward recursion, value iteration, policy evaluation, occupancy, Monte Carlo simulation), `fast2048.py` (the same rules on a 64-bit bitboard compiled with numba, about 50 times faster, checked move by move against the reference implementation by `test_2048.py`), `predict2048.py` (TD(0), n-step TD and Monte Carlo prediction), `shaping2048.py` (the rules-of-thumb reward of Section 6.2), `rl2048.py` (tabular control agents and the exact evaluator), `rlfast2048.py` (the same agents on the compiled engine, with the table in a hash table, for the 4×4 board),`dqn2048.py` (the DQN of the final test, with the tutorial's schedule) and `dqn_gpu2048.py` (the same agent with many games at once on the GPU, with the perceptron or the convolutional network). Every number and figure of this report is produced by a script in `experiments/` and written to `results/`:

Script	Produces
`dp_figures.py`	Section 5.4: Tables 3, 4, Figures 2a–3b
`dp_value_iteration.py`	Section 5.1: Table 2, Figure 1 (2 minutes)
`exp_reward.py`, `plot_reward.py`	Section 6.2: Tables 5, 6, Figure 4 (15 min)
`exp_prediction.py`	Section 6.3: Figure 5 (50 runs, under a minute)
`exp_tabular_2x3.py`, `plot_tabular.py`	Section 6.4: Table 7, Figure 6 (90 runs, 45 min on 26 cores)
`exp_tabular_grid.py`, `plot_schedule.py`	Figures 7, 8 (130 runs)
`exp_tabular_4x4.py`, `plot_tabular_4x4.py`	Section 6.5: Table 8, Figure 9 (90 runs, 10 min, up to 3 GB of memory per run; with `check`, the port on 2×3)
`eval_4x4.py`	rules of thumb on the 4×4 board, Table 10
`exp_dqn.py`	Section 6.6: the tutorial's schedule (6 runs in parallel on the CPU, 50 min)
`exp_dqn_gpu.py`, `plot_dqn_gpu.py`	Section 6.6: many games on the GPU (6 runs, 18 min; with `net=cnn`, the convolutional network: 2 runs, 95 min), Table 9, Figures 10, 11
`export_viewer.py`	data of the policy viewer (best policy of every method, retrained)
`bench_engine.py`	throughput of the two engines

The notebook `2048_DP.ipynb` walks through Part 1 interactively, and `viz2048.py` animates any of the policies playing. The page `viewer/index.html`, which opens in any browser, shows the best policy of every method of this report playing: on the 2×3 board each policy is a table over the decision states, retrained with the configuration and the seed of its best run (the training is deterministic, so its exact win probability is the one reported), and the page shows at every move the optimal win probability V^* and the exact value Q^* of every move; on the 4×4 board it plays the rules of thumb, the tabular methods of Section 6.5 (the 50 000 boards each table met most often, the corner rule elsewhere) and the deep Q-networks of the final test. The page is published at `https://edoardocaproni.github.io/2048/`.

References

[1] Ahmed Abdelkader, Aditya Acharya, and Philip Dasler. 2048 without new tiles is still hard. In *8th International Conference on Fun with Algorithms (FUN 2016)*, volume 49 of *LIPIcs*, pages 1:1–1:14, 2016. Preprint: On the complexity of slide-and-merge games, arXiv:1501.03837.

[2] Dimitri P. Bertsekas. *Reinforcement Learning and Optimal Control*. Athena Scientific, 2019.

[3] Gabriele Cirulli. 2048. `https://github.com/gabrielecirulli/2048`, 2014.

[4] Hung Guei. *On Reinforcement Learning for the Game of 2048*. PhD thesis, National Yang Ming Chiao Tung University, 2022. arXiv:2212.11087.

[5] Wojciech Jaśkowski. Mastering 2048 with delayed temporal coherence learning, multistage weight promotion, redundant encoding, and carousel shaping. *IEEE Transactions on Computational Intelligence and AI in Games*, 10(1):3–14, 2018. arXiv:1604.05085.

[6] Stefan Langerman and Yushi Uno. Threes!, Fives, 1024!, and 2048 are hard. In *8th International Conference on Fun with Algorithms (FUN 2016)*, volume 49 of *LIPIcs*, pages 22:1–22:14, 2016. Journal version: Theoretical Computer Science 748, 17–27, 2018.

[7] Rahul Mehta. 2048 is (PSPACE) hard, but sometimes easy, 2014. arXiv:1408.6315.

[8] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.

[9] Adam Paszke and Mark Towers. Reinforcement learning (DQN) tutorial. PyTorch Tutorials, `https://docs.pytorch.org/tutorials/intermediate/reinforcement_q_learning.html`. Accessed November 2026.

[10] Alexey Slizkov. Computational bounds for the 2048 game, 2023. arXiv:2303.07266.

[11] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

[12] Marcin Szubert and Wojciech Jaśkowski. Temporal difference learning of N-tuple networks for the game 2048. In *IEEE Conference on Computational Intelligence and Games (CIG)*, pages 1–8, 2014.

[13] Robert Xiao. 2048 AI: expectimax optimisation with a bitboard engine. `https://github.com/nneonneo/2048-ai`, 2014.

[14] Kun-Hao Yeh, I-Chen Wu, Chu-Hsuan Hsueh, Chia-Chuan Chang, Chao-Chin Liang, and Han Chiang. Multistage temporal difference learning for 2048-like games. *IEEE Transactions on Computational Intelligence and AI in Games*, 9(4):369–380, 2017.